

# Comentários sobre os Testes — Analisador Léxico/Sintático

Disciplina: Teoria dos Compiladores | Prof.<sup>a</sup> Alessandra Hauck

---

Este documento apresenta os testes realizados no compilador desenvolvido com JFlex e CUP, observando o comportamento do Analisador Léxico (**scanner.flex**) e do Analisador Sintático (**parser.cup**) para cada expressão solicitada. A gramática utiliza recursividade **à direita** para as operações de multiplicação, divisão e módulo, e recursividade **à esquerda** para soma e subtração.

## Teste 1: 1+3;

Saída do compilador:

```
1 + 1.0
3.0
+
= 4.0
```

Resultado: = 4.0

Comentário:

Expressão de adição simples. O scanner reconhece os tokens NUMBER(1), PLUS, NUMBER(3) e SEMI. O parser reduz cada NUMBER para *term* → *factor*, depois aplica *expr* → *expr PLUS factor*. O resultado 4.0 está correto ( $1 + 3 = 4$ ). Observa-se que os números são impressos primeiro pelo scanner (ao serem lidos) e depois pelo parser ao reduzir a regra *term* ::= NUMBER.

## Teste 2: 1-2;

Saída do compilador:

```
1 - 1.0
2.0
-
= -1.0
```

Resultado: = -1.0

Comentário:

Expressão de subtração. Tokens: NUMBER(1), MINUS, NUMBER(2), SEMI. A regra *expr* → *expr MINUS factor* é aplicada, resultando em  $1 - 2 = -1.0$ . O valor negativo é representado corretamente como Double.

## Teste 3: 1;

Saída do compilador:

```
1 1.0
= 1.0
```

Resultado: = 1.0

Comentário:

Expressão com apenas um número. O scanner retorna NUMBER(1) e SEMI. A cadeia de reduções é: NUMBER → *term* → *factor* → *expr* → *expr\_part*. Nenhum operador é disparado. O

resultado impresso é 1.0, confirmando que o tipo base da gramática é Double.

#### Teste 4: $1 * 2;$

Saída do compilador:

```
1 * 1.0
2.0
*
= 2.0
```

Resultado: = 2.0

Comentário:

Expressão de multiplicação. Tokens: NUMBER(1), TIMES, NUMBER(2), SEMI. A regra *factor*  $\rightarrow$  *term TIMES factor* (recursividade à direita) é aplicada. Resultado:  $1 * 2 = 2.0$ . Com apenas dois operandos, a direção da recursividade não altera o resultado neste caso.

#### Teste 5: $1 * 2 * 3;$

Saída do compilador:

```
1 * 1.0
2 * 2.0
3.0
*
*
= 6.0
```

Resultado: = 6.0

Comentário:

Multiplicação encadeada com três operandos. Pela **recursividade à direita** da regra *factor*  $\rightarrow$  *term TIMES factor*, a expressão é avaliada como  $1 * (2 * 3)$ . A saída mostra que o operador '\*' do par  $(2 * 3)$  é impresso antes do '\*' externo, comprovando a associatividade à direita. Resultado:  $1 * (2 * 3) = 6.0$ .

#### Teste 6: $1 - 2 - 3;$

Saída do compilador:

```
1 - 1.0
2 - 2.0
-
3.0
-
= -4.0
```

Resultado: = -4.0

Comentário:

Subtração encadeada. A regra *expr*  $\rightarrow$  *expr MINUS factor* usa **recursividade à esquerda**, portanto a expressão é avaliada como  $(1 - 2) - 3$ . O '-' entre 1 e 2 é reduzido primeiro:  $(1 - 2) = -1$ , depois  $(-1 - 3) = -4.0$ . Isso contrasta com a multiplicação: a subtração associa à esquerda, enquanto a multiplicação associa à direita nesta gramática.

---

**Conclusão:** Os testes confirmam que a gramática está funcionando corretamente. A **recursividade à direita** nas operações de multiplicação/divisão implica associatividade à direita (ex.:  $1*2*3 = 1*(2*3)$ ), enquanto a **recursividade à esquerda** na soma/subtração implica associatividade à esquerda (ex.:  $1-2-3 = (1-2)-3$ ). Ambos os arquivos **scanner.flex** e **parser.cup** foram ajustados para trabalhar com valores do tipo **Double** (em vez de Integer), garantindo precisão em operações com decimais.